DSA I (CSE 2216 & CSE 2215)

Linear Search and Binary Search

Asif Abrar, Lecturer of CSE

UIU

Searching Algorithms – Overview

- Searching finds the position of a target element in a list.
- Two basic comparison-based searching techniques:

  Linear Search and **Binary Search**.
- Binary Search requires the array to be sorted.

Linear Search – Intuition

- Check each element one by one from left to right.
- Stop when the element is found.
- Works on unsorted arrays.
- Worst Case: target is at the end or not present.
- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

Linear Search – Pseudocode

Algorithm:

1. For $i = 0$ to $n - 1$:
   - If `arr[i] == key`, return `i`
2. If end of array reached $\rightarrow$ key not found

Observation: No need for sorting; scans sequentially.

Linear Search – Key Algorithm (C++)

```
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i; // Found
    }
    return -1; // Not found
}
```

Linear Search – Simulation

Array: 7 2 9 4 1 5
Search Key: 4

Step	Checked Element	Status
1	7	Not match
2	2	Not match
3	9	Not match
4	4	Found at index 3

Result: Key found at index 3.

Binary Search – Intuition

- Works only on sorted arrays.
- Repeatedly divides the search space into two halves.
- Compare key with middle element:
  - If equal $\rightarrow$ found
  - If smaller $\rightarrow$ search left half
  - If larger $\rightarrow$ search right half
- Time Complexity: $O(\log n)$
- Space Complexity: $O(1)$

Binary Search – Pseudocode

Algorithm:

1. Set `low = 0`, `high = n-1`
2. While `low <= high`:
   - `mid = (low + high) / 2`
   - If `arr[mid] == key` → return mid
   - Else if `key < arr[mid]` → high = mid - 1
   - Else → low = mid + 1
3. Not found → return -1

Binary Search – Key Algorithm (C++)

```cpp
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == key)
            return mid;
        else if (key < arr[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1; // Not found
}
```

Binary Search – Simulation

Sorted Array: `2 4 7 10 15 20`
Search Key: `10`

Step	Low–High	Mid	Status
1	0–5	2	$7 < 10 \rightarrow$ Move Right
2	3–5	4	$15 > 10 \rightarrow$ Move Left
3	3–3	3	10 Found

Result: Key found at index 3.

Comparison Between the Two

Property	Linear Search	Binary Search
Approach	Sequential Scan	Divide and Conquer
Best Case	$O(1)$	$O(1)$
Worst Case	$O(n)$	$O(\log n)$
Needs Sorted?	No	Yes
In-place	Yes	Yes
Use Case	Small or unsorted lists	Large sorted lists

Binary Search – Lower/Upper Range Concept

Key Idea: Instead of looking for exact match, binary search can be used to find:

- **Lower Bound**: first index where
  `arr[i] >= key`
- **Upper Bound**: first index where
  `arr[i] > key`

Important: Search space is the *index range*, not the data itself. We repeatedly shrink the range using mid.

Lower Bound – Intuition

- Find the **first position** where the element is $\geq$ key.
- Even if the exact key does not exist, this returns the correct insertion position.
- Useful in:
  - Frequency counting
  - Range queries
  - Searching repeated values
- Time Complexity: $O(\log n)$

Lower Bound – Code (C++)

```
int lowerBound(int arr[], int n, int key) {
    int low = 0, high = n; // high = n (not n-1)

    while (low < high) {
        int mid = (low + high) / 2;

        if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid;
    }
    return low; // first index where arr[i] >= key
}
```

Lower Bound – Simulation

Array: 2 4 4 4 7 9 11
key = 4

Step	Low–High	Mid	Action
1	0–7	3	arr[3] = 4 >= key → high = 3
2	0–3	1	arr[1] = 4 >= key → high = 1
3	0–1	0	arr[0] = 2 < key → low = 1

Lower Bound Index = 1

Upper Bound – Intuition

- Find the **first index** where element is **greater than key**.
- This gives the position where elements $\leq$ key end.
- Useful for:
  - Counting occurrences: `count = upperBound - lowerBound`
  - Insertion position after duplicates
- Time Complexity: $O(\log n)$

Upper Bound – Code (C++)

```
int upperBound(int arr[], int n, int key) {
    int low = 0, high = n;

    while (low < high) {
        int mid = (low + high) / 2;

        if (arr[mid] <= key)
            low = mid + 1;
        else
            high = mid;
    }
    return low; // first index where arr[i] > key
}
```

Upper Bound – Simulation

Array: 2 4 4 4 7 9 11
key = 4

Step	Low–High	Mid	Action
1	0–7	3	$arr[3] = 4 <= key \rightarrow low = 4$
2	4–7	5	$arr[5] = 9 > key \rightarrow high = 5$
3	4–5	4	$arr[4] = 7 > key \rightarrow high = 4$

Upper Bound Index = 4

Lower vs Upper Bound – Comparison

Feature	Lower Bound	Upper Bound
Condition	arr[i] >= key	arr[i] > key
Returns	First occurrence / insert pos	End of duplicates
Useful for	Insert before duplicates	Insert after duplicates
Count freq	Yes (UB-LB)	Yes
Works on sorted?	Yes	Yes

Linear Search: Best, Worst and Average Case

Linear Search scans elements one by one.

Best Case: $\Omega(1)$

- Target is found at the **first index**.
- Only 1 comparison.

Worst Case: $O(n)$

- Target is at the **last index** OR not present.
- All n elements must be checked.

Average Case: $\Theta(n)$

- Target is equally likely to appear at any position.
- Expected comparisons $= \frac{n+1}{2}$.

Binary Search: Best, Worst and Average Case

Binary Search repeatedly halves the search space (sorted array).

Best Case: $\Omega(1)$

- The **middle element** is the target.
- Only 1 comparison.

Worst Case: $O(\log n)$

- Target is at the **deepest level** of the recursion/tree.
- Every step halves the range $\rightarrow$ about $\log_2 n$ steps.

Average Case: $\Theta(\log n)$

- Expected position is somewhere among all levels of the binary search tree.
- Average depth half of worst-case depth $\rightarrow$ still logarithmic.

Why Average Case is $\Theta(n)$ for Linear Search

Assumption: Target is equally likely to appear at any of the n positions.

- If the target is at position $1 \rightarrow 1$ comparison
- If the target is at position $2 \rightarrow 2$ comparisons
- ...
- If the target is at position $n \rightarrow n$ comparisons

$$\text{Average comparisons} = \frac{1 + 2 + 3 + \cdots + n}{n}$$

Using sum of first n integers:

$$\frac{n(n+1)/2}{n} = \frac{n+1}{2}$$

Therefore: Average Case

$$\Theta(n)$$

Why Average Case is $\Theta(\log n)$ for Binary Search

Binary Search splits the array into halves:

$$n \to \frac{n}{2} \to \frac{n}{4} \to \cdots \to 1$$

Worst case depth: $\log_2 n$

Average case idea:

- **Elements can be found at any level of the search tree.**
- **Most nodes are in the lower (deeper) levels.**
- **The average depth is roughly the midpoint of the tree height.**

$$\textbf{Average depth} \approx \frac{1}{2}\log_2 n$$

Constant factors ignored in asymptotic notation:

$$\Theta\left(\frac{1}{2}\log n\right) = \Theta(\log n)$$

Simple Task for Students

Task: Write a C++ program that performs both Linear Search and Binary Search. Print the number of comparisons performed in each case.

Sample Input: 2 5 7 9 11 15 (key = 9)

Expected Output:

```
Linear Search: Found at index 3 (Comparisons: 4)
Binary Search: Found at index 3 (Comparisons: 2)
```

Hint: Use mid calculation and comparison counting.

Thank You!

Questions?